

Advanced Text-to-Image Generation

Digital Egypt Pioneers Initiative - DEPI Project Documentation April 2025

Team Members

Aya Mohamed Youssef

Basant Ragb Said Abdel Hafez

Mostafa Mahmoud Mohamed

Abdelrahman Yasser

CONTENT:

- **Abstract**
- **Introduction**
 - Background
 - Motivation
 - Problem
- **Methodology & Implementation**
 - Data Collection
 - Data Preprocessing
 - Model Architecture
 - Models
 - Approaches
- **Integration**
- **Tools**
- **Results**
- **Limitations**
- **Conclusion**

Abstract

The Advanced Text-to-Image Generation project aims to explore and implement cutting-edge generative AI techniques to convert textual descriptions into high-quality, visually coherent images.

By harnessing the capabilities of Generative Adversarial Networks (GANs) and diffusion models, the project seeks to bridge the gap between natural language understanding and visual representation.

This fusion of language and vision enables the creation of AI systems capable of interpreting complex textual inputs and translating them into detailed visual outputs, paving the way for applications in creative industries, design automation, accessibility tools, and beyond.

Introduction

Background

This project focuses on generating medical images directly from textual descriptions using the RoCo-radiology dataset. The RoCo dataset, derived from Radiology Objects in Context, is a large and comprehensive collection of radiological images paired with detailed captions. By leveraging these text-image pairs, the project aims to develop models that can create accurate and meaningful medical images based solely on the input text. This approach highlights the potential of text-to-image generation to enhance medical imaging by synthesizing visual content from natural language descriptions.

Motivation

Our objective in targeting the medical field is to enhance diagnostic accuracy, support medical education, and accelerate research efforts. By leveraging advanced machine learning techniques, this project aims to drive innovation in clinical practices, improve healthcare outcomes, and make medical data more accessible and interpretable for healthcare professionals and patients alike.

Problem

The main challenge of this project is to generate accurate and meaningful medical images from textual descriptions in the radiology domain. The dataset consists of radiology images paired with detailed captions, which require careful preprocessing such as cleaning the text by removing stopwords and applying lemmatization, as well as standardizing the images by resizing and converting them to a single-channel format. To address this challenge, we use a model that combines diffusion techniques with a U-Net architecture. The diffusion model gradually reconstructs images from noise, while the U-Net extracts important features to ensure the generated images

align closely with the input text. Additionally, fine-tuning the model on domain-specific data helps improve the quality and relevance of the generated images. This approach aims to bridge the gap between natural language descriptions and visual medical data, enabling better support for diagnostic and educational purposes.

Methodology: Data Collection

The Radiology Objects in Context (ROCO) dataset is a large-scale, multimodal medical imaging dataset. It features images sourced from publications available through the PubMed Central Open Access FTP mirror, automatically identified as either radiology or non-radiology and non-compound. Each image is provided with a downloadable link and an associated caption. In addition, the dataset includes keywords extracted from the captions, along with their corresponding UMLS Semantic Types (SemTypes) and Concept Unique Identifiers (CUIs). All images in ROCO are paired with descriptive captions.

The dataset comprises over **81,183** radiology images spanning various medical imaging modalities such as CT, ultrasound, X-ray, fluoroscopy, PET, mammography, MRI, and angiography.

Structure:

The dataset consists of three columns: image, image_id, and caption.

image image · width (px) 438~54716.5%	image_id string · lengths 10100%	caption string · lengths 5~29193.1%
	ROCO_00029	Post-Treatment MRIAxial T1-weighted, contrast-enhanced image showing the same lesion as Figure 3.
	ROCO_00030	Abdominal CT scan showing pancreatic pseudocyst
	ROCO_00032	Axial contrast-enhanced computed tomography image of the abdomen 3 months after anti tuberculous treatment showing...
	ROCO_00033	Lytic lesions (arrows) involving the humerus.

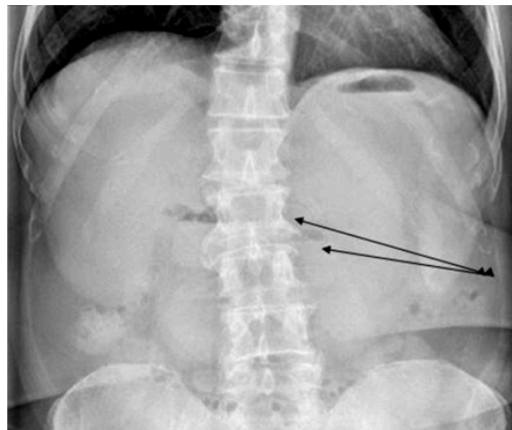
To streamline our research and enhance model training. The dataset was organized by splitting it according to imaging modality.

As there was no dedicated column specifying the scan type, the modality information was extracted directly from the image captions using keyword-based filtering. Identifying mentions of specific scan types, such as X-ray, CT, MRI, and Ultrasound.

The images were grouped into separate subsets based on modality. Our primary focus was X-ray images, which were extracted through this method and then preprocessed for further analysis and model training.

Sample of the dataset:

Plain abdomen x-ray: Multiple air levels at the mid-abdomen (arrows), no radiopaque shadow, and no air under the diaphragm.



Methodology: Data preprocessing

Image Preprocessing Steps:

- **Resizing Images**

All images were resized to a fixed resolution to ensure uniform input dimensions for the model. This helps maintain consistency across the dataset and reduces computational complexity.

- **Converting to Single-Channel (Grayscale)**

Images were transformed to grayscale, reducing them to a single channel. This is particularly suitable for medical imaging tasks, such as X-ray analysis, where color information is not essential and grayscale captures all relevant details.

Captions Preprocessing Steps:

- **Stopword Removal**

Common words that do not carry significant meaning (such as "*the*", "*is*", "*and*") were removed from the captions to reduce noise and focus on the more informative parts of the text.

- **Lemmatization**

Words were reduced to their base or root form (e.g., "*running*" → "*run*") to normalize the text and treat different forms of a word as the same token.

- **Addition of Start and End Tokens**

Special tokens (e.g., <start> and <end>) were added to the beginning and end of each caption to help the model learn when to start and stop generating text during training.

- **Tokenizer Preparation**

A tokenizer was built to convert the text captions into sequences of integers, where each integer represents a specific word in the vocabulary.

- **Caption-to-Sequence Conversion**

Using the tokenizer, all captions were transformed into numerical sequences so they could be fed into the model for training.

- **Embedding Matrix Creation**

An embedding matrix was constructed to map each word in the vocabulary to a dense vector representation, capturing semantic relationships between words and improving the model's understanding of language.

Methodology: Models

The development of the text-to-image model began with an exploration of Generative Adversarial Networks (GANs). However, it became clear that GANs, particularly image-to-image models, were not well-suited for generating images directly from textual descriptions. To address this limitation, the focus shifted towards diffusion models, which are better suited for text-to-image tasks due to their ability to progressively refine image details during the generation process.

Given the complexity and scale of diffusion models, fine-tuning them initially presented significant challenges, especially concerning memory and computational demands. To overcome these obstacles, a custom base model was constructed from scratch. This model incorporated an autoencoder for efficient feature extraction and a set of diffusion model components, including cross-attention layers that allow the alignment of textual information with image features during generation. Despite these innovations, the results from this approach remained insufficient in terms of detail and fidelity.

The performance of the model was significantly improved through fine-tuning. By adapting the model specifically to the text-to-image generation task, its ability to produce accurate and high-quality images from textual descriptions was enhanced.

The final model, after successful fine-tuning, integrated these components cohesively, ensuring efficient generation of detailed and semantically correct images from textual inputs.

Methodology: Approaches

This project explores three distinct generative modeling approaches for medical image synthesis from textual descriptions. Each method incorporates unique architectural components tailored to the challenges of X-ray image generation.

1. GAN-Based Generation

A custom Generative Adversarial Network (GAN) was trained using text embeddings as conditioning input. While capable of learning basic structural patterns, this approach struggled to produce anatomically accurate or diagnostically useful X-ray images. Limitations were observed in both resolution and fine medical detail.

2. Custom Diffusion Model

A diffusion-based architecture was implemented with added attention mechanisms and conditional text inputs. This model showed improvements in structural coherence and alignment with textual prompts. However, the generated images still lacked visual detail and accuracy, particularly in more complex anatomical areas.

3. Fine-Tuned Stable Diffusion (Best Results)

The most effective results were achieved by fine-tuning a pre-trained **Stable Diffusion model** on a domain-specific X-ray dataset. This approach integrates:

- A **U-Net** backbone for denoising with skip connections that preserve spatial features.
- **Variational Autoencoders (VAEs)** to compress input images into a latent space, improving representation and control.

- **Text conditioning** via CLIP-based embeddings to align image synthesis with medical prompts.

Diffusion models operate by gradually introducing noise into an image and learning to reverse this process to generate realistic outputs from random noise. The fine-tuned Stable Diffusion model excelled at producing high-quality, semantically accurate, and medically plausible X-ray images. The combined use of VAEs and U-Net enables the model to maintain both global context and fine anatomical detail, crucial in the medical imaging domain.

Methodology: GAN-Based Generation

The GAN-based approach for text-to-image generation begins by preparing the dataset and text captions. First, the dataset is split into training and testing sets (95% training, 5% testing). The captions are tokenized using a pre-trained tokenizer, and an embedding matrix is loaded to represent the vocabulary. The captions are then converted to sequences and padded to ensure consistent input length for the model. For the image data, the X-ray images are loaded and preprocessed by resizing them to 256x256 pixels and normalizing pixel values. The images are stored in a directory, and any missing files are skipped during preprocessing.

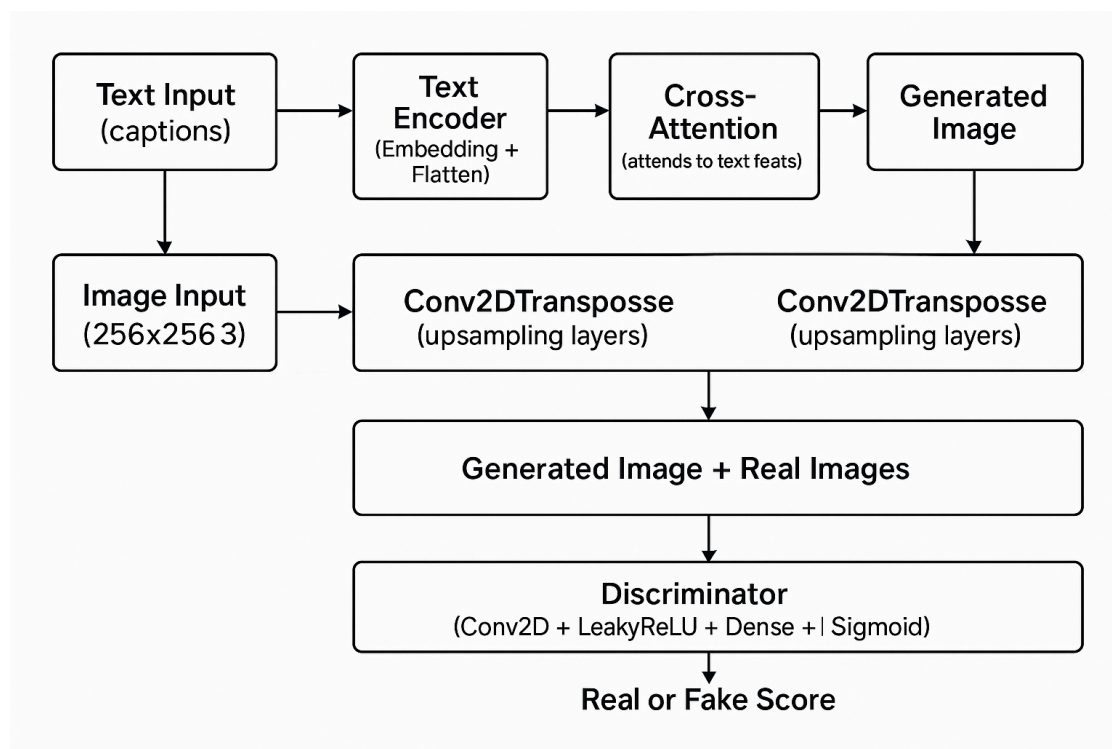
For the model architecture, a custom GAN is built with a generator that incorporates cross-attention layers. The text data is first encoded using a simple text encoder that converts the captions into feature vectors through embedding and flattening. These text features are then used as queries, keys, and values in the cross-attention layer to allow the model to focus on relevant textual information when generating the image. The image input is then processed through a series of Conv2DTranspose layers to upsample and refine the generated image. The result is a GAN model that takes both textual descriptions and images as inputs to generate high-quality X-ray images.

The discriminator of the GAN is designed to distinguish between real and fake X-ray images. It consists of convolutional layers with LeakyReLU activations to progressively extract features from the input image. The output is a single sigmoid unit that predicts whether the image is real or fake. The discriminator is trained with real and fake labels, where real images are labeled with 0.9 and fake images with 0.1, to help stabilize training. The generator is trained to fool the discriminator by minimizing its loss function, which uses binary cross-entropy.

The training loop involves alternating between training the discriminator and the generator. During each epoch, the discriminator is trained with both real images from the dataset and fake images generated by the generator. The discriminator's loss is calculated as the average of its losses on real and fake images. After updating the discriminator, the generator is trained through the GAN model, which combines the generator and discriminator into a single model that outputs a probability of whether a generated image is real or fake. The generator's loss is calculated and minimized to improve its ability to produce realistic images.

After training for a set number of epochs, the trained generator, discriminator, and GAN model are saved for future use.

AttnGAN Flow Diagram



Step-by-Step Summary Table AttnGAN Approach:

Step	Component	Input(s)	Output(s)	Description
1	Text Encoder	Tokenized captions(max_length, h,)	Flattened text embedding(batch_size, max_length * embedding_dim)	Embeds and flattens text sequences into dense vectors.
2	Cross-Attention Layer	Flattened text embedding (query, key, value)	Attention output(batch_size, seq_len, embed_dim)	Applies attention over text features to highlight important semantic parts.
3	Image Input	Image tensor(256, 256, 3)	Same shape	Initial image or noise input for the generator.
4	Conv2DTranspose Layers	Image input	Upsampled image tensor	Upsamples and refines the image features progressively.
5	Generator Output	Output of Conv2DTranspose	Generated image(256, 256, 3)	The final generated image is conditioned on text and the input image.
6	Discriminator	Real or generated image(256, 256, 3)	Real/Fake probability(batch_size, 1)	Classifies images as real or fake.
7	Training Loop	The batch of real images and captions	Updates discriminator and generator weights	Alternately, the discriminator should be trained on real/fake images, and the generator should be trained to fool the discriminator.

Results:

```
# Generate images
generated_images = generator.predict([test_captions_sample, test_images_sample])

# Plot results
fig, axes = plt.subplots(num_samples, 2, figsize=(10, num_samples * 3))

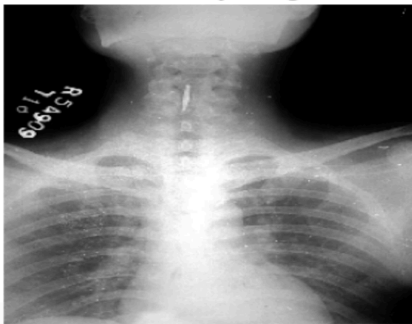
for i in range(num_samples):
    axes[i, 0].imshow(test_images_sample[i]) # Original X-ray image
    axes[i, 0].set_title("Real Image")
    axes[i, 0].axis("off")

    axes[i, 1].imshow(generated_images[i]) # Generated Image
    axes[i, 1].set_title("Generated Image")
    axes[i, 1].axis("off")

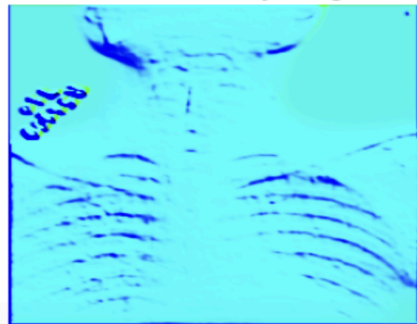
plt.show()
```

1/1 ————— 0s 49ms/step

Real X-ray Image



Generated X-ray Image



```
In [ ]: # Generate multiple images for the same text
num_samples = 5
fig, axes = plt.subplots(1, num_samples, figsize=(15, 3))

for i in range(num_samples):
    generated_image = generator.predict([np.expand_dims(test_caption, axis=0), np.expand_dims(test_image, axis=0)])
    generated_image = np.clip(generated_image[0], 0, 1)

    axes[i].imshow(generated_image)
    axes[i].axis("off")
    axes[i].set_title(f"Generated {i+1}")

plt.show()
```

1/1 ————— 0s 36ms/step

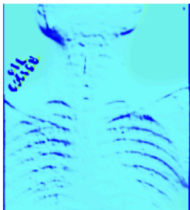
1/1 ————— 0s 35ms/step

1/1 ————— 0s 35ms/step

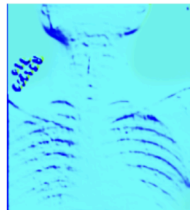
1/1 ————— 0s 37ms/step

1/1 ————— 0s 34ms/step

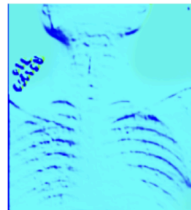
Generated 1



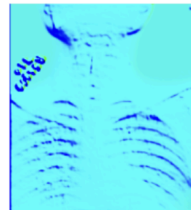
Generated 2



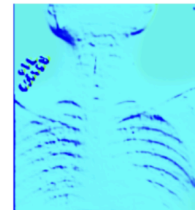
Generated 3



Generated 4



Generated 5



Methodology: Custom Diffusion Model

In the first approach of building a diffusion model from scratch, the ResolutionHandler module plays a crucial role in managing changes in spatial resolution while preserving temporal information via time embeddings. This module is designed to process input feature maps with dynamic channel support and resolution adjustment, making it highly adaptable across different stages of a diffusion network. It applies two convolutional layers with intermediate Group Normalization and SiLU activation, ensuring stable feature transformation. A key component is the optional integration of time embeddings, projected and added to the feature map to incorporate timestep-specific context. The module also ensures residual learning through a shortcut connection that adapts based on input-output channel mismatch. When the current resolution differs from the target, bilinear interpolation is applied to both the main and residual branches, ensuring spatial consistency. This design ensures robust processing across varying feature map dimensions and channels, a fundamental requirement for diffusion models that evolve data representations over multiple denoising steps.

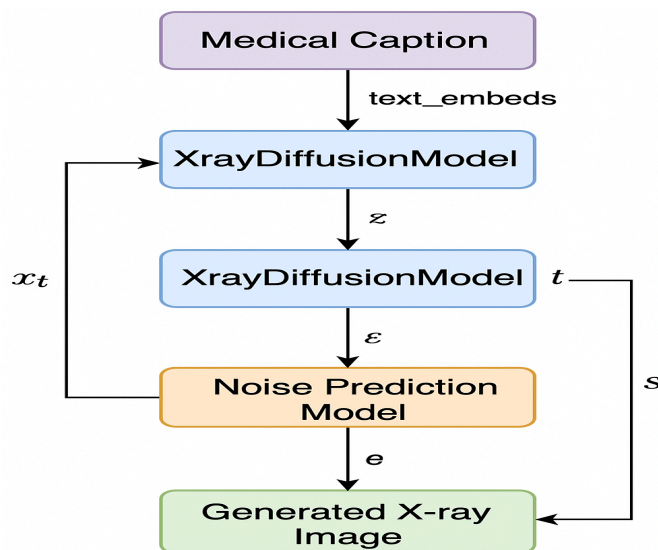
Building upon the foundational ResolutionHandler, the complete model architecture introduces several key components tailored for medical image generation. The SinusoidalPositionEmbeddings module generates smooth and continuous time-step embeddings, enabling the model to be aware of the diffusion step during training and inference. The ResBlock modules serve as the core building blocks of the U-Net, incorporating time conditioning and residual connections, with adaptive handling of mismatched input channels. Meanwhile, the CrossAttention module allows the model to integrate external context, specifically, textual medical descriptions, by computing attention between latent image features and text embeddings.

These components are orchestrated within the XrayUNet class, a hierarchical U-Net architecture that captures both global structure and fine-grained details.

During the forward pass, the input image is processed through a time-aware encoder path with optional attention at selected resolutions, followed by a bottleneck region combining residual and attention modules. The decoder then upsamples the latent representation while incorporating skip connections and further conditioning on the text.

Extending from the MedicalTextEncoder design, this module plays a pivotal role in conditioning the image generation process on medical captions. The embedding layer transforms discrete word indices into continuous vector representations, optionally initialized with pre-trained embeddings to infuse prior semantic knowledge.

The bidirectional GRU processes these embeddings sequentially from both directions, enabling the capture of contextual information from the entire sentence, essential for understanding complex medical terminology and structure. By concatenating the forward and backward hidden states and



passing the result through a projection layer, the encoder compresses the rich temporal information into a fixed-length vector. This final vector, representing the global semantic summary of the medical caption, is then used as the conditioning input to the diffusion model's CrossAttention layers.

The integration of bidirectional recurrence ensures that both past and future word dependencies are leveraged, improving the alignment between textual prompts and generated X-ray image features.

Continuing from the XrayDiffusionModel, this class integrates a noise prediction model into a denoising diffusion probabilistic framework to synthesize realistic X-ray images conditioned on medical text embeddings. The training process involves a forward diffusion phase where Gaussian noise is incrementally added to the image over a fixed number of steps, simulating the gradual corruption of data. During training, the model learns to reverse this process by predicting the original noise at each step, using the noisy image and corresponding text features.

In the generation phase, the model leverages DDIM (Denoising Diffusion Implicit Models) sampling, which allows for faster inference by taking fewer, non-randomized steps while maintaining high-quality outputs. Notably, the sampling loop includes classifier-free guidance, a powerful technique that interpolates between unconditional and conditional generations to better align outputs with the conditioning input, in this case, medical captions. This guidance mechanism allows fine control over the degree of conditioning via the `cfg_scale` parameter, effectively steering the output toward clinically relevant features described in the prompt.

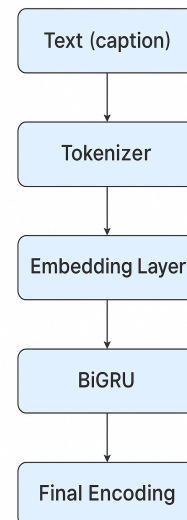
By predicting the noise component in each timestep, the model iteratively refines a noisy sample back into a coherent image.

The final output is normalized to a $[0, 1]$ range, ensuring compatibility with image visualization pipelines. The elegance of this approach lies in its combination of semantic conditioning through `text_embeds`, probabilistic modeling via diffusion, and architectural modularity, allowing plug-in models like U-Net with attention.

After initializing the `text_encoder` (an instance of `MedicalTextEncoder`), the function returns a tuple containing:

1. `embedding_matrix` – This is either:
 - The embedding matrix is generated from the tokenizer, if provided, usually from pretrained embeddings, or
 - None if no tokenizer is passed, in which case the model uses a randomly initialized embedding layer.
2. `text_encoder` – The actual model that will convert a batch of tokenized caption sequences into fixed-length feature vectors. This encoder:
 - Maps input word indices to word embeddings.
 - Passes those embeddings through a bidirectional GRU to capture sequence-level information.
 - Applies a linear projection to reduce or unify the output dimensionality.
 - Outputs a `[batch_size, hidden_dim]` tensor per sequence, which becomes the conditioning vector for the diffusion model.

Setup of Medical Text Embeddings



The Variational Autoencoder (VAE) is a neural network that compresses images into a smaller, meaningful representation (latent space) and then reconstructs them. It has an encoder that outputs a mean and variance for sampling, and a decoder that recreates the image. During training, it minimizes two losses: how accurately it reconstructs the image (reconstruction loss) and how well the latent space stays organized (KL divergence loss).

After training, the encoder can convert images into compact latent codes. These codes can then be combined with text embeddings in a **MixedEncoderDiffusion** model, which uses both image features (from the VAE) and text to generate new, realistic images.

Then training the Variational Autoencoder (VAE) on X-ray images, followed by a diffusion model conditioned on text captions. The VAE is used to encode the images, and the diffusion model uses both image encodings and text embeddings for generation. After training, the models and tokenizer are saved for later use.

Defining a class `XRayDiffusionGenerator` that uses a diffusion-based model to generate X-ray images from text prompts. The class initializes the necessary components, including tokenizers, text encoders, UNet, VAE, and scheduler, with support for custom or default models. It tokenizes the input text, encodes the prompt using the text encoder, and uses a denoising process with the UNet model to generate an image. The `generate_image` method performs the image generation through multiple inference steps, guided by a classifier-free guidance scale. Finally, the generated image is decoded from latent space and returned as a PIL image. A utility function `generate_xray_from_text`, wraps this process for easy usage.

Highlight of the Stable Diffusion Approach:

- ResolutionHandler Module
 - Manages spatial resolution changes while preserving temporal info via time embeddings.
 - Uses two convolution layers with Group Normalization and SiLU activation.
 - Adds projected time embeddings to feature maps for timestep context.
 - Employs residual connections with channel adaptation.
 - Applies bilinear interpolation when resolution changes to maintain spatial consistency.
- Core Model Components
 - SinusoidalPositionEmbeddings: Generates smooth time-step embeddings for diffusion awareness.
 - ResBlock Modules: Core U-Net blocks with time conditioning and residual connections.
 - Cross-Attention Module: Integrates medical text embeddings with image features for conditioning.
 - XrayUNet: Hierarchical U-Net combining encoder, bottleneck (residual + attention), and decoder with skip connections and text conditioning.
- MedicalTextEncoder
 - Embeds medical captions into continuous vectors.
 - Uses bidirectional GRU to capture full sentence context.
 - Projects concatenated forward and backward states into fixed-length semantic vectors.
 - Provides conditioning vectors for CrossAttention in the diffusion model.
- Diffusion Model Training & Generation
 - Forward diffusion adds Gaussian noise progressively to images.
 - The model learns to predict noise at each step, conditioned on noisy image and text embeddings.

- Uses DDIM sampling for faster inference with fewer steps.
- Applies classifier-free guidance to balance unconditional and conditional generation, controlled by `cfg_scale`.
- Outputs normalized images compatible with visualization.
- Variational Autoencoder (VAE)
 - Encodes images into latent space (mean and variance).
 - Decoder reconstructs images minimizing reconstruction and KL divergence losses.
 - Latent codes combined with text embeddings in `MixedEncoderDiffusion` for generation.
- X-Ray Diffusion Generator Class
 - Initializes tokenizer, text encoder, U-Net, VAE, and scheduler.
 - Tokenizes and encodes text prompts.
 - Uses denoising diffusion with classifier-free guidance to generate a latent image.
 - Decodes latent to final image (PIL format).

Step-by-Step summary Table For Custom Diffusion Model Approach

Component	Purpose	Key Features
ResolutionHandler	Manage resolution changes + time context	Conv layers + GroupNorm + SiLU + time embeddings + residual + bilinear interpolation
SinusoidalPositionEmbeddings	Time-step embedding for diffusion awareness	Smooth continuous embeddings
ResBlock	U-Net building block	Time conditioning + residual connections
CrossAttention	Integrate text context into image features	Attention between latent image and text embeddings
MedicalTextEncoder	Encode medical captions	Embedding layer + bidirectional GRU + projection
Variational Autoencoder (VAE)	Compress/reconstruct images	Encoder outputs mean/variance + decoder + KL + reconstruction loss
XrayUNet	Hierarchical U-Net for image generation	Encoder + bottleneck (residual + attention) + decoder + skip connections + text conditioning
Diffusion Model	Noise prediction and denoising	Forward diffusion + noise prediction + DDIM sampling + classifier-free guidance
XRayDiffusionGenerator	Orchestrates the generation pipeline	Initializes components, runs text encoding, diffusion, and decoding

Results :

Prompt: *"A human chest x-ray"*

Generated X-ray



Methodology: Fine-Tuning Diffusion Model

The third approach to fine-tune Stable Diffusion is implemented by training only the UNet model while keeping the VAE and CLIP text encoder frozen. The `CLIPTextModel` is loaded and frozen using `requires_grad_(False)`, ensuring its weights are not updated during backpropagation. Similarly, the `AutoencoderKL` VAE is also frozen to retain its ability to encode X-ray images into latent representations and decode them back, without altering its learned distribution. During training, each batch consists of X-ray images and associated medical captions. The captions are tokenized using the CLIP tokenizer, and their embeddings are extracted via the frozen text encoder. Images are encoded into latents by the VAE, and noise is added to simulate the diffusion process. The UNet then learns to predict this noise based on the latent, timestep, and text embeddings. The model is trained using MSE loss between predicted and actual noise. Validation, logging (with W&B and MLflow), and image quality metrics like CLIP score and FID are integrated into the loop. This setup enables domain-specific fine-tuning for medical X-rays, optimizing only the UNet for generating high-quality, caption-aligned images, while leveraging the strengths of the pretrained components.

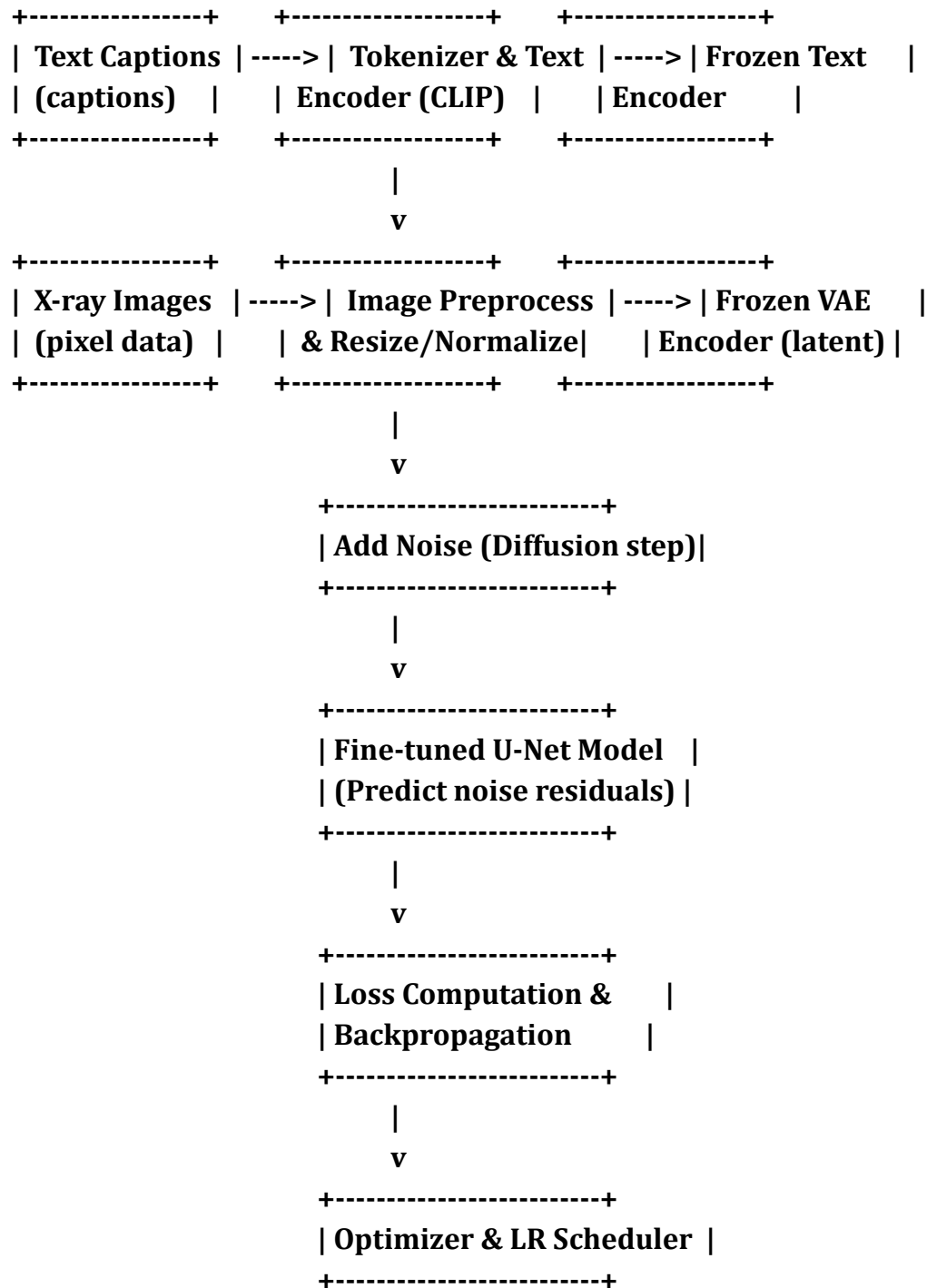
Once training is complete, the fine-tuned model can be used for inference via the `generate_xray_from_text` function, which loads the trained checkpoint using the `StableDiffusionPipeline`, moves it to the appropriate device, and generates an X-ray image from a medical text prompt using classifier-free guidance. The function supports saving the generated image and allows for additional controls like adjusting inference steps, resolution, and adding a negative prompt to refine output quality, making it practical for generating high-fidelity, medically aligned synthetic X-rays from textual descriptions.

Fine-Tuning Diffusion Model: Key Points

- Model and Dataset Setup
 - Start from a pre-trained Stable Diffusion model (e.g., "CompVis/stable-diffusion-v1-4").
 - Prepare a custom dataset of X-ray images paired with medical text captions.
 - Use a tokenizer to encode text captions and image transforms to preprocess images.
- Model Components
 - Freeze the text encoder and VAE to keep their pre-trained weights fixed.
 - Fine-tune the U-Net component of the diffusion model to adapt to the X-ray domain.
 - Use a noise scheduler (DDPMScheduler) to manage the diffusion noise schedule.
- Training Procedure
 - For each batch:
 - Encode text captions using the frozen text encoder.
 - Encode images into latent space using the frozen VAE encoder.
 - Add noise to latents according to a random timestep.
 - Use U-Net to predict noise residuals conditioned on noisy latents and text embeddings.
 - Compute mean squared error loss between predicted and actual noise.
 - Backpropagate loss and update U-Net weights.
 - Use gradient accumulation, mixed precision (fp16), and cosine learning rate scheduler.

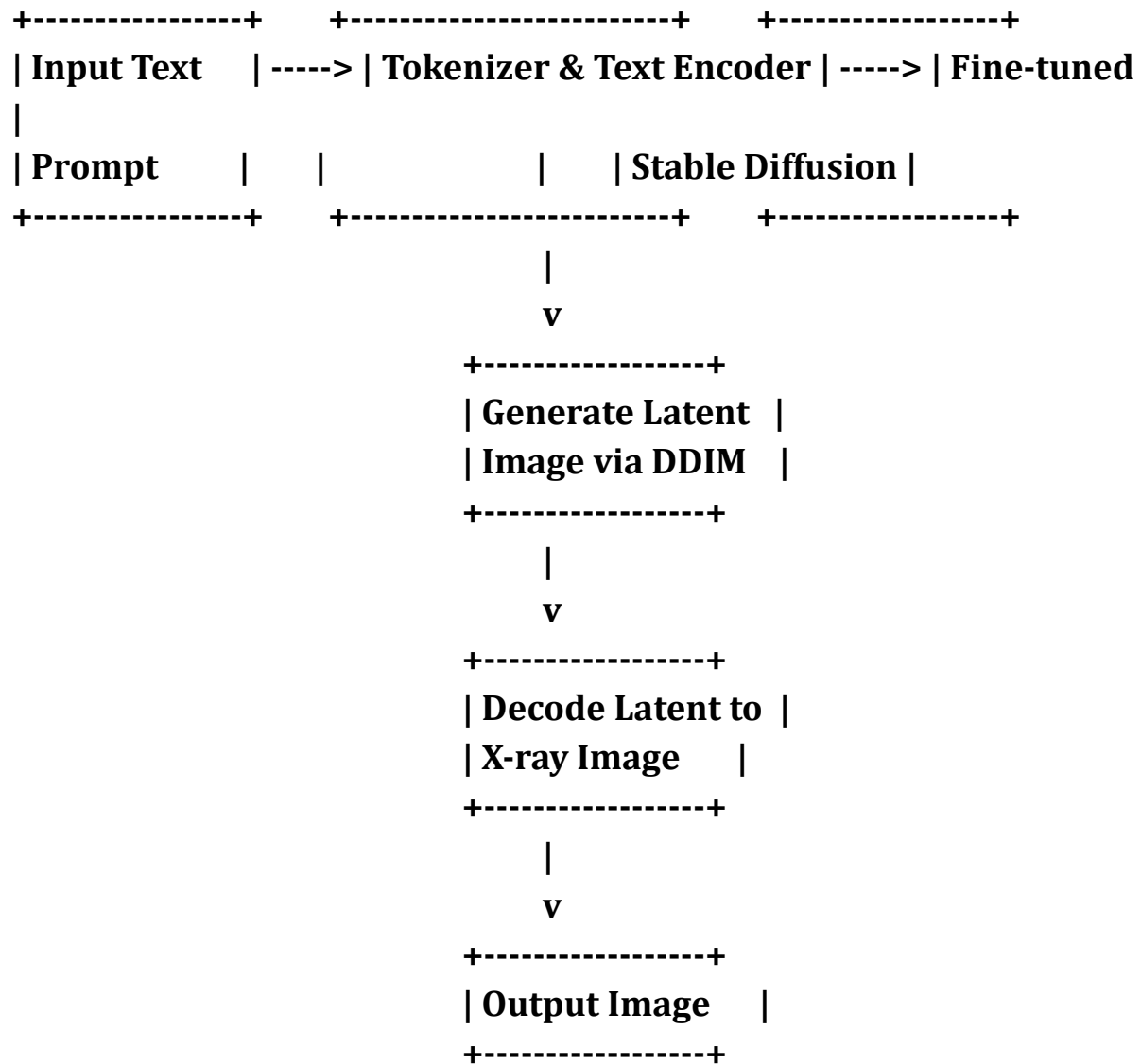
- Log training metrics (loss, learning rate, gradient norm) with Weights & Biases and MLflow.
- Periodically evaluate on the validation set and generate sample images.
- Compute metrics like CLIP score and Frechet Inception Distance (FID) to assess image quality.
- Save model checkpoints at regular intervals.
- Inference
 - Load fine-tuned model checkpoint.
 - Use the Stable Diffusion pipeline to generate X-ray images from input text prompts.
 - Control generation quality with parameters like the number of inference steps and guidance scale.
 - Optionally save generated images.
- Evaluation and Monitoring
 - Use the CLIP model to compute the similarity between generated images and prompts.
 - Use FID to measure distribution similarity between generated and real images.
 - Visualize samples during training for qualitative feedback.

Flow Diagram Description:



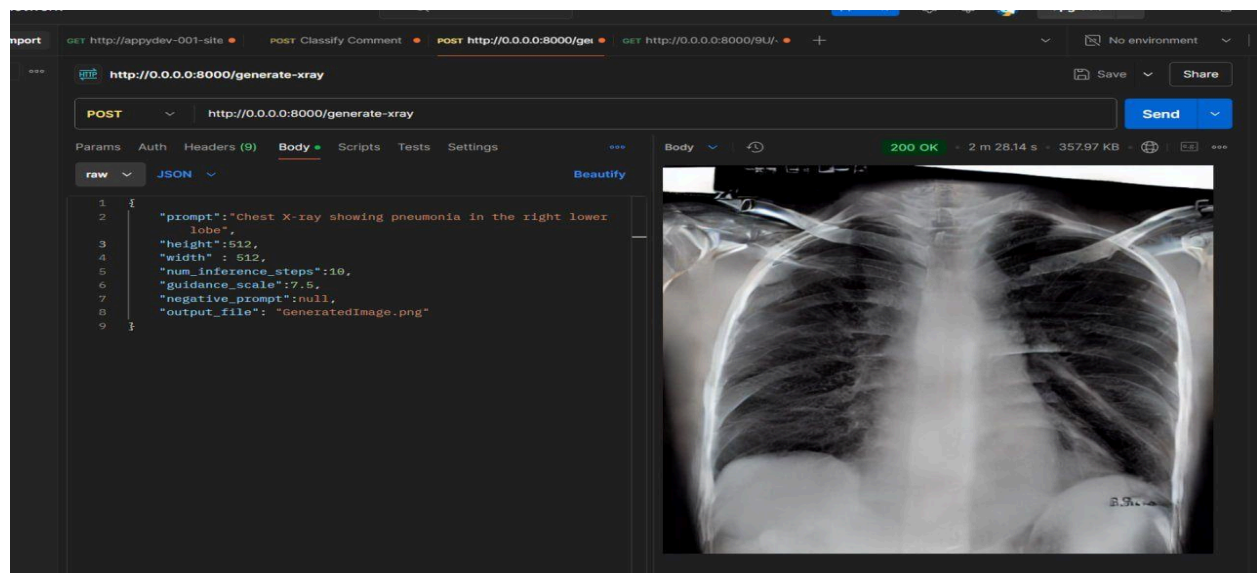
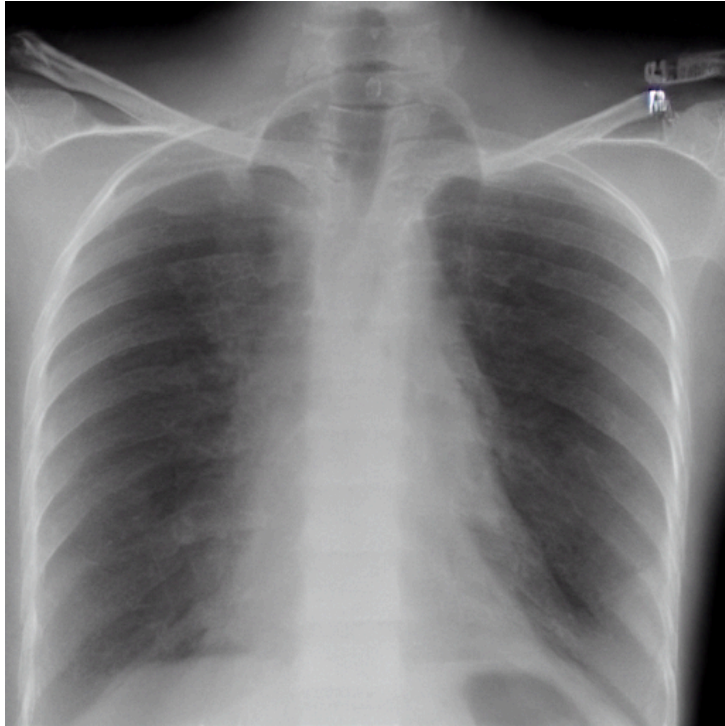
Training loop iterates over batches and epochs, logging metrics and saving checkpoints.

Inference:



Results:

Prompt : *"Chest X-ray showing pneumonia in the right lower lobe"*



Training Phase:

To train a text-to-image diffusion model using the diffusers library with PyTorch and hardware accelerators, the process typically involves setting up a U-Net-based denoising network alongside a noise scheduler such as DDPM Scheduler. The captions from the RoCo dataset are preprocessed and converted into embeddings, which serve as conditional inputs to guide image generation. Images are resized and converted to a single channel to match the model's input expectations. During training, random noise is added to the images at varying timesteps, and the model learns to predict and remove this noise conditioned on the text embeddings. The accelerate library is used to simplify and optimize training across different hardware setups, supporting multi-GPU and mixed precision training. This setup allows efficient, large-scale training of a conditional diffusion model that can generate medical images aligned with textual descriptions, enhancing interpretability and application in healthcare AI systems.

Integration: Overview

In our approach, we focused on building a robust and trackable machine learning pipeline for X-ray image generation using PyTorch. While TensorBoard is a common choice for visualizing training metrics through a GUI, we opted not to use it since our implementation was fully based on PyTorch, and instead leveraged tools better aligned with our stack, such as Weights & Biases (wandb) for experiment tracking.

Throughout training, we monitored key metrics including training loss, validation loss, CLIP score, and gradient norm to assess model performance and stability.

For deployment, we exposed the model through a FastAPI-based REST API using well-defined request and response schemas, ensuring structured and validated communication. Finally, the entire application was containerized using Docker, allowing us to run the service locally or deploy it to a virtual machine seamlessly, ensuring environment consistency and ease of deployment.

Tools

Gradio:

To provide a seamless and accessible user experience, we integrated the X-ray generation pipeline with a Gradio-based web interface. Gradio is a Python library that enables quick deployment of machine learning models through interactive browser-based UIs with minimal code overhead. Our integration approach focuses on modularity and clarity, separating the interface logic from the model functionality. The Gradio interface collects user inputs such as the prompt, negative prompt, image dimensions, inference steps, guidance scale, model path, and optional output file. These parameters are passed to a wrapper function, which preprocesses the inputs (e.g., ensuring image dimensions are compatible with the model) and then calls the core `generate_xray_from_text` function. The output image is displayed along with a save status message, providing immediate visual feedback. This approach allows for efficient experimentation and easy deployment while maintaining a clean and maintainable codebase.

Docker

To support consistent deployment across different environments, we containerized the application using Docker. The provided Dockerfile sets up a lightweight Python 3.10 environment, installs the necessary dependencies from `requirements.txt`, copies the application code, and launches the API server using `uvicorn` on port 8000. The working directory is set to `/app`, and the container runs the FastAPI application (`xray_inference_api:app`) to handle inference requests. This containerized setup ensures reproducibility, simplifies deployment, and makes the solution portable across development, testing, and production environments.

Wandb

To monitor and manage our model training and inference experiments, we integrated Weights & Biases (wandb) into the pipeline. wandb is a powerful tool for tracking machine learning experiments, visualizing model performance, and collaborating across teams. During training or inference, key metrics such as loss curves, guidance scale effects, inference step changes, and output image samples can be automatically logged to the wandb dashboard. This allows for real-time visualization and comparison across multiple runs, aiding in hyperparameter tuning and model evaluation. Furthermore, wandb's artifact system can be used to version model checkpoints and generated images, ensuring reproducibility and traceability across the development workflow.

FastAPI(Pydantic)

Schema of Request and Response:

Defining the **request and response schemas** for the X-ray image generation API using **Pydantic**, which provides data validation and structured data models in Python, especially when working with FastAPI.

Request Schema (XrayRequest)

The XrayRequest class describes the input data expected by the API when a user wants to generate an X-ray image. It includes:

- **prompt (*str*):** A required text description that guides the X-ray image generation.
- **height (*int*, default=512):** The height of the generated image. It should be a multiple of 8.

- `width` (*int*, default=512): The width of the generated image. It should also be a multiple of 8.
- `num_inference_steps` (*int*, default=50): The number of steps the model should take during inference—higher values may improve image quality.
- `guidance_scale` (*float*, default=7.5): Controls how strongly the model follows the prompt—a higher value typically leads to more accurate generations.
- `negative_prompt` (*Optional[str]*): An optional prompt to steer the model away from certain characteristics (e.g., "blurry", "low quality").
- `output_file` (*Optional[str]*): An optional path where the generated image can be saved.

This schema ensures the API receives structured and validated data for generating X-ray images.

Response Schema (XrayResponse)

The XrayResponse class defines the structure of the data returned by the API after image generation:

- `image_base64 (str)`: The generated X-ray image encoded in Base64 format. This makes it easy to transfer the image via JSON without saving it to a file.
- `save_message (str)`: A message indicating whether the image was saved successfully or not, depending on whether an `output_file` path was provided.

This response format allows the client to both visualize the generated image and get feedback about saving status.

MLflow Tracking:

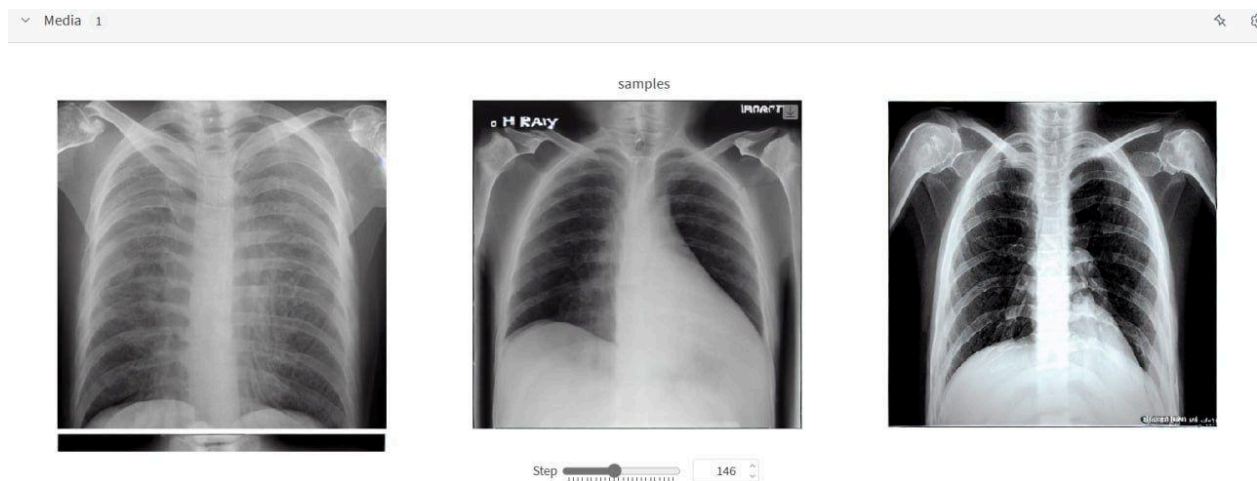
MLflow Tracking was used to log experiments, including parameters, metrics, and artifacts across different runs.

It helped in comparing model versions and maintaining a reproducible workflow.

The tracking server enabled centralized management of all training experiments and results.

Results: Stable Diffusion Finetuning

The model was trained for two epochs, with each epoch comprising multiple training steps to ensure thorough learning. Given the complexity of the architecture, which includes a U-Net backbone, Variational Autoencoders (VAEs), and a diffusion framework, the training process required careful tuning and resource management. At each step, the model's ability to generate realistic X-ray images was evaluated using a fixed textual prompt. This consistent evaluation ensured that the model progressively learned to produce coherent and medically relevant outputs aligned with the given descriptions. Below is a sample of the images generated by the model, demonstrating its capability to synthesize plausible X-ray visuals based on the provided text input.



Clip score:

The graph presents the evolution of the CLIP score throughout the training process, indicating how well the generated images align with their corresponding textual descriptions.



Over the course of 2500 training steps, the CLIP score remains relatively stable, fluctuating within the range of approximately 0.33 to 0.35. This suggests a consistent performance in generating semantically relevant images. A temporary drop is observed around step 1400, where the score dips below 0.31, likely due to momentary instability or changes in learning dynamics. However, the model quickly recovers, and by step 2400, the CLIP score reaches approximately 0.3418, reflecting a strong correlation between the text prompts and the generated visuals. Overall, the results indicate that the model is effectively learning to generate images that align well with their descriptive text inputs.

Gradient Norm:

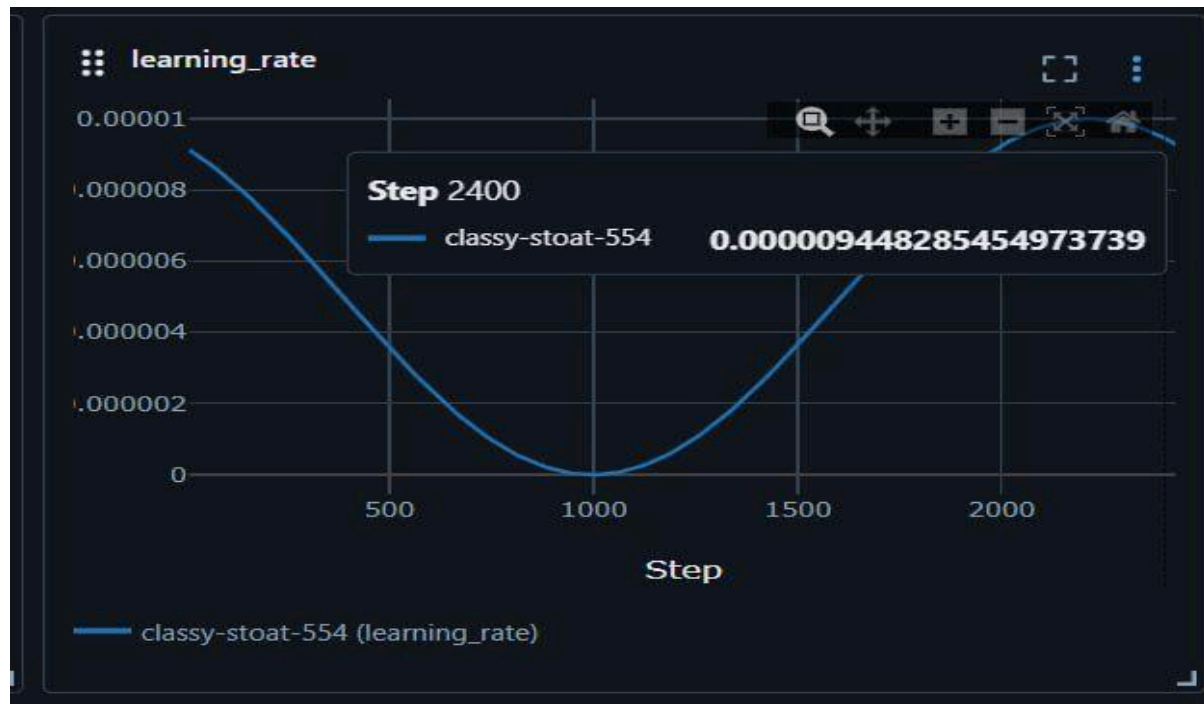
The gradient norm, which reflects the magnitude of the weight updates during training, generally exhibits a fluctuating but relatively stable pattern throughout the training process.



However, a notable and substantial spike is observed around step 1750, reaching a significantly higher value. This sudden increase in the gradient norm suggests a moment of instability or a sharp change in the model's error landscape at that particular training step. While the gradient norm appears controlled for most of the training, this isolated peak warrants further scrutiny to understand its cause and potential impact on the overall training outcome.

Learning Rate:

The learning rate, which dictates the step size for updating the model's weights during training, follows a dynamic schedule.

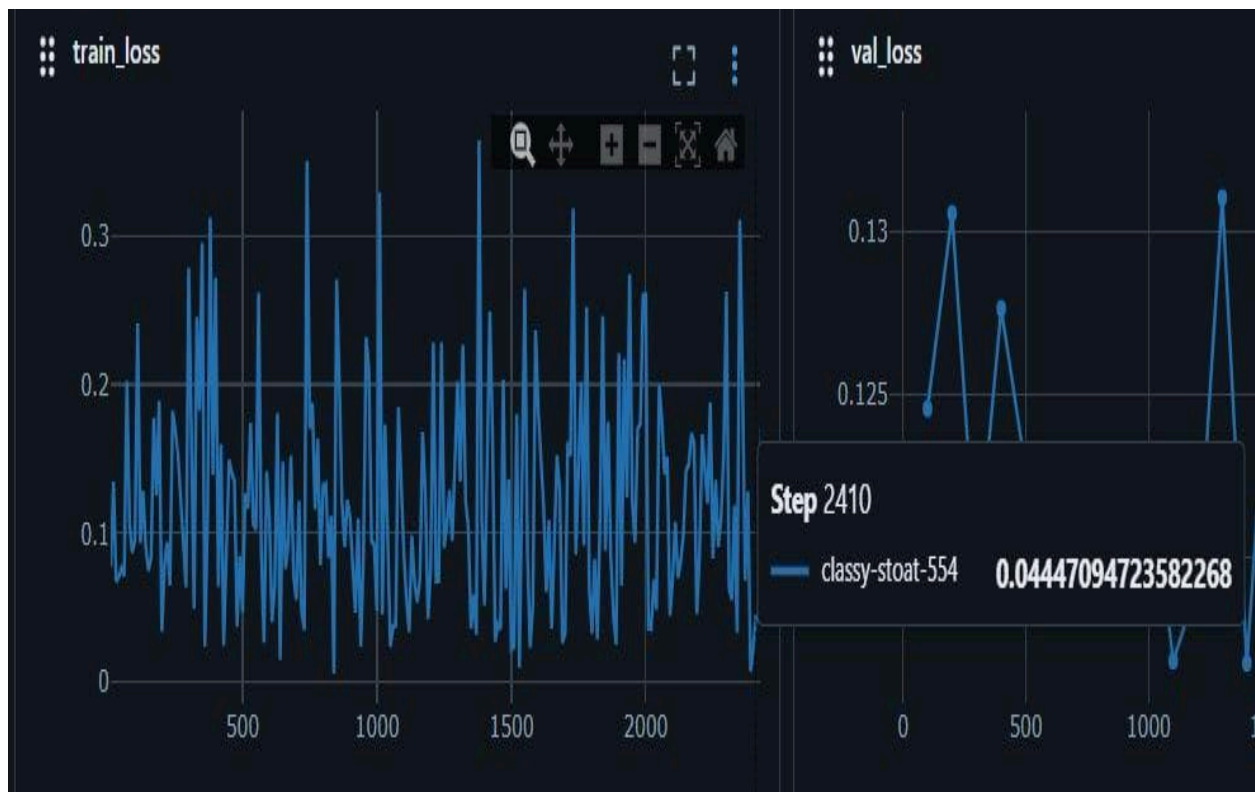


It

starts at a relatively higher value and then undergoes a significant decrease, reaching its minimum around step 1000. Following this trough, the learning rate gradually increases again towards the later stages of training. This cyclical learning rate schedule is a strategy often employed to initially allow for larger adjustments in the model's parameters, then fine-tune them more precisely as training progresses, and potentially escape local minima by increasing the learning rate later on. At step 2400, the learning rate is approximately 0.0000094.

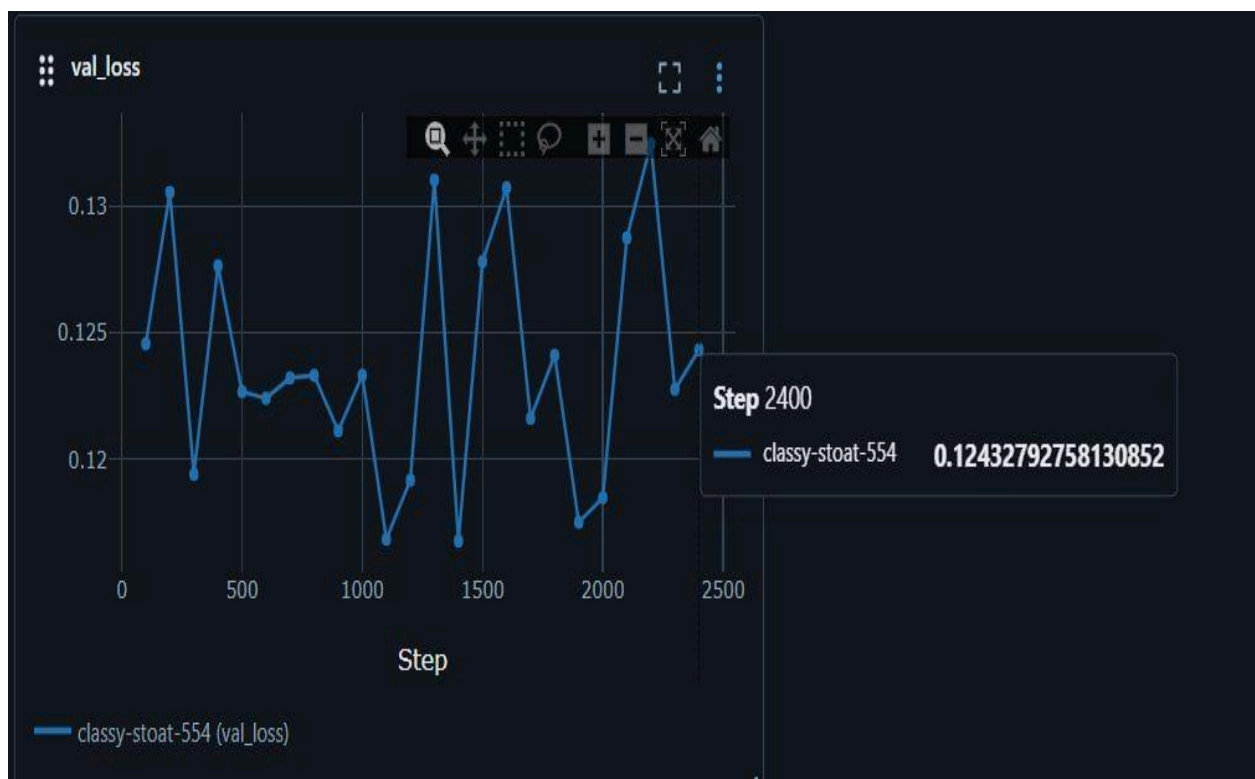
Training Loss

The training loss jumps around a lot, showing inconsistent performance on the training data. In contrast, the validation loss, which measures performance on new data, has smoother ups and downs. At step 2410, the validation loss is about 0.0447. The different patterns suggest the model's learning on seen and unseen data isn't perfectly aligned.



Validation Loss:

The validation loss, which tracks the model's performance on unseen data, exhibits fluctuations with noticeable peaks and troughs throughout the training. These ups and downs indicate changes in how well the model generalizes to new examples as training progresses. At step 2400, the validation loss is approximately 0.1243. This value, along with the overall trend of the validation loss, provides insight into the model's ability to generalize and helps in identifying potential overfitting or underfitting.



Limitations

1. Model Size and Memory Constraints:

- The initial challenge was the large size of the diffusion model, which created memory and computational limitations. The sheer size of the model made fine-tuning difficult and resource-intensive, especially when dealing with high-dimensional data like text and images.

2. Suboptimal Initial Results:

- Despite building the model from scratch, the early results were not satisfactory. This could be attributed to the complexity of aligning text and image data effectively using cross-attention layers and the initial lack of enough fine-tuning.

3. Training Time:

- Training large-scale diffusion models requires extensive computational resources and time, which slowed the progress at the beginning. The iterative nature of training such models also contributed to longer periods before we could see improvements in the output.

4. Fine-Tuning Challenges:

- Fine-tuning, although eventually successful, posed a challenge in terms of hyperparameter optimization and adapting the model to the specific needs of the text-to-image task. The fine-tuning process required careful monitoring and adjustments to ensure the model did not overfit or underperform.

5. Integration Complexity:

- Integrating the various components (autoencoder, diffusion model, cross-attention layers) required careful design and troubleshooting to ensure all parts of the model functioned cohesively during the text-to-image generation process.

6. Fluctuating Loss Curves:

- Both the training and validation loss plots exhibited high variability across steps, indicating instability during training. This could be due to noisy gradients, suboptimal learning rates, or challenges in aligning complex medical captions with visual representations. The inconsistent loss trends made it harder to determine convergence and required constant monitoring.

7. CLIP Score Inconsistency:

- The CLIP score, which measures how well the generated images align with their corresponding text prompts, showed irregular dips, especially around step 175. Such fluctuations suggest the model occasionally struggled to maintain strong semantic alignment between image and text, which could impact the reliability of the generated outputs in clinical contexts.

8. Learning Rate Scheduling Sensitivity:

- The learning rate followed a cyclical schedule, but its dramatic changes may have contributed to unstable training behavior. The model might have been sensitive to learning rate extremes, causing either stagnation (when too low) or erratic updates (when too high).

9. Limited Epochs:

- Training was conducted for only two epochs, which might not have been sufficient for a model of this complexity to fully learn the intricate relationships between radiology images and their textual descriptions. This limitation likely contributed to the early suboptimal results and unstable performance indicators.

Conclusion

This project demonstrates the potential of combining diffusion models with U-Net architectures for text-to-image generation in the medical field, specifically using radiology data from the RoCo dataset. Despite facing challenges such as large model size, memory constraints, training instability, and fine-tuning complexity, significant progress was made. Through iterative development and optimization, the model was able to generate X-ray images conditioned on medical captions with promising quality. The integration of various components—autoencoder, cross-attention layers, and diffusion mechanisms—enabled effective learning of the relationship between text and medical imagery. Although there were limitations, particularly in training stability and resource requirements, the results lay a strong foundation for future work in generating and interpreting medical images, with potential applications in clinical education, diagnostics, and research.

